Designing a **key-value store** is a classic system-design problem. The basic API is simple, but the real design depends on scale, durability, consistency, and latency requirements.

1. Start with the API

At minimum:

```
PUT(key, value)
GET(key)
DELETE(key)
```

Common extensions:

```
PUT(key, value, ttl)
GET_VERSION(key, version)
COMPARE_AND_SET(key, expected_version, new_value)
BATCH_GET(keys)
BATCH_PUT(items)
SCAN(prefix)
```

Example:

http

```
PUT /kv/customer:123
{
  "value": {
    "name": "Alice",
    "status": "ACTIVE"
  },
  "ttl_seconds": 86400
}
```

The key should usually be treated as an opaque byte string. The value may be JSON, binary, text, or serialized objects.

---

2. Define the requirements

A reasonable interview assumption might be:

- Billions of keys
- Value size: a few bytes to several MB
- Very high read and write throughput
- `GET` latency under 10 ms
- Data must survive machine failures

- Horizontal scaling
- Configurable replication
- Optional TTL
- No complex joins or relational queries
- Point lookup by exact key is the primary operation

Important questions include:

- Is strong consistency required?
- Are range scans required?
- Is the workload read-heavy or write-heavy?
- What is the maximum value size?
- Can stale reads be tolerated?
- Must writes remain available during network partitions?
- Is this an in-memory cache or a durable database?

These answers strongly affect the storage engine and replication design.

---

3. High-level architecture

```
| SSTables or B-tree storage          |
├─────────────────────────────────────┤
| Compaction / cleanup                |
└─────────────────────────────────────┘
```

The major components are:

1. Client routing
2. Partitioning
3. Storage engine
4. Replication
5. Consistency
6. Failure recovery
7. Rebalancing
8. Monitoring and operations

---

4. Partitioning the data

The store must distribute keys across many machines.

Option A: Hash partitioning

```
partition = hash(key) % number_of_partitions
```

Example:

Python Run

```python
partition_id = hash("customer:123") % 1024
```

Advantages:

- Even data distribution
- Fast point lookup
- Simple routing

Disadvantages:

- Changing the number of partitions can move many keys
- Range scans are difficult
- Hot keys can still overload one partition

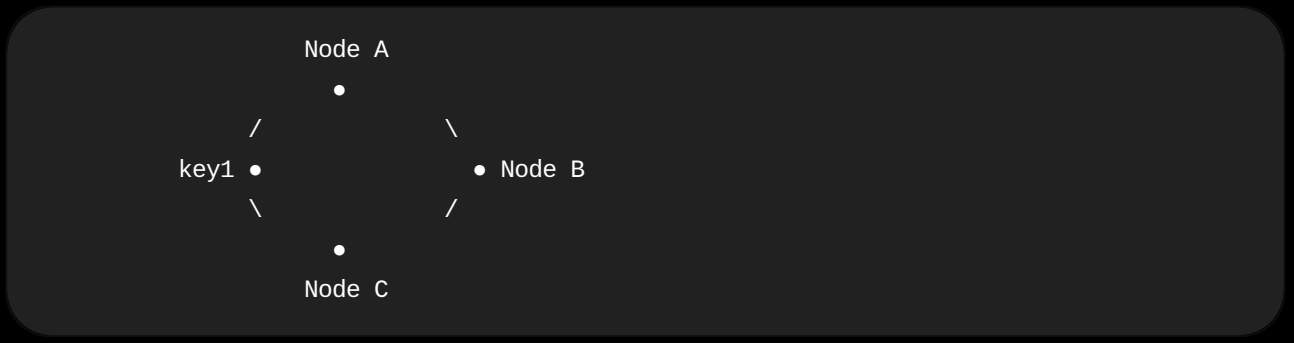
In practice, create many **logical partitions**, such as 1,024 or 16,384, and assign those partitions to physical nodes.

```
Key
 |
 ▼
hash(key)
 |
 ▼
Logical partition
 |
 ▼
Physical server
```

This separates data partitioning from machine count.

Option B: Consistent hashing

Place nodes and keys on a hash ring.



A key is stored on the next node clockwise.

Advantages:

- Adding or removing nodes moves only part of the data
- Useful in Dynamo-style systems

Disadvantages:

- More complicated balancing
- Virtual nodes are usually required
- Range queries remain difficult

Option C: Range partitioning

Examples:

```
Partition 1: A–F
Partition 2: G–M
Partition 3: N–S
Partition 4: T–Z
```

Advantages:

- Supports ordered scans

- Efficient prefix and range queries

Disadvantages:

- Uneven traffic distribution
- Sequential keys can create hot partitions
- Split logic is required

For a general key-value store dominated by exact-key lookup, hash partitioning is usually the best starting point.

---

5. Storage engine

There are two common choices.

Option A: LSM tree

An LSM-tree design is good for write-heavy systems.

Write path:

```
PUT(key, value)
      |
      ▼
Write-ahead log
      |
      ▼
MemTable in memory
      |
      ▼
Immutable MemTable
      |
      ▼
SSTable on disk
      |
      ▼
Background compaction
```

Write sequence

1. Append the change to the write-ahead log.
2. Update an in-memory sorted structure called a MemTable.
3. Acknowledge the write after durability requirements are met.
4. When the MemTable becomes full, flush it to disk as an SSTable.
5. Merge SSTables later through compaction.

Read sequence

```
GET(key)
   │
   ├── Check cache
   ├── Check active MemTable
   ├── Check immutable MemTables
   └── Search SSTables
          └── Bloom filter first
```

Bloom filters avoid unnecessary disk reads:

```
Bloom filter says "definitely absent"
    → skip the SSTable

Bloom filter says "possibly present"
    → search the SSTable index
```

Advantages:

- High sequential write throughput
- Good for heavy ingestion
- Easy append-oriented durability

Disadvantages:

- Compaction creates write amplification
- Reads may check multiple SSTables
- Disk space temporarily increases during compaction

RocksDB, Cassandra, HBase, and LevelDB use variations of this model.

Option B: B-tree or B+ tree

A B-tree-based engine updates pages closer to their final location.

Advantages:

- Predictable point reads
- Good range scans
- Lower read amplification

Disadvantages:

- More random writes
- Page splitting and update overhead
- Usually lower raw write throughput than an LSM tree

For a distributed key-value system with heavy writes, an LSM-tree is a common choice.

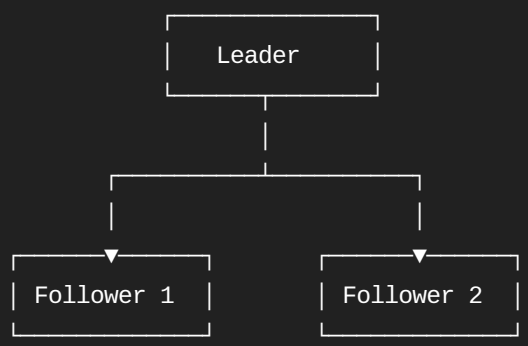
---

6. Write path

For a replicated partition:

```
Client
  |
  ▼
Partition leader
  |
  ├── Append to local WAL
  ├── Replicate to followers
  ├── Update MemTable
  └── Return success
```

With replication factor 3:



Acknowledgement policies may be:

```
ONE    → return after one replica stores the write
QUORUM → return after a majority stores the write
ALL    → return after every replica stores the write
```

Trade-off:

Policy	Latency	Durability	Availability
ONE	Lowest	Lower	Highest
QUORUM	Medium	Strong	Medium
ALL	Highest	Highest	Lowest

For a strongly consistent system, the leader normally waits for a majority.

---

7. Read path

A strongly consistent read usually goes to the leader:

```
Client → router → partition leader → value
```

A lower-latency eventually consistent read may go to any replica:

```
Client → nearest replica → possibly stale value
```

The node searches in this order:

```
1. In-memory cache
2. MemTable
3. Immutable MemTables
4. SSTable Bloom filters
5. SSTable sparse index
6. SSTable data blocks
```

Frequently accessed values may be stored in an LRU or LFU block cache.

---

8. Replication model

Leader-follower replication

Each partition has one leader.

```
Partition 42
├── Leader: Node A
├── Follower: Node B
└── Follower: Node C
```

Writes go to the leader. Followers replicate the leader's log.

Advantages:

- Straightforward ordering
- Easier strong consistency
- Simple conflict handling

Disadvantages:

- Leader can become a bottleneck
- Leader election is needed
- Temporary unavailability during election

Consensus protocols such as Raft can manage:

- Leader election
- Replicated log
- Membership changes
- Commit ordering

Leaderless replication

Clients or coordinators write to multiple replicas.

With:

```
N = number of replicas
W = write acknowledgements required
R = replicas read
```

A common condition is:

```
W + R > N
```

Example:

```
N = 3
W = 2
R = 2
```

This provides overlapping read and write quorums, but conflict resolution is still required.

Possible conflict techniques:

- Last-write-wins timestamp
- Version vectors
- Application-level merge
- CRDTs

For an interview, leader-follower replication with Raft is generally easier to explain when strong consistency is required.

---

9. Consistency guarantees

You should explicitly state the guarantees.

Strong consistency

After a successful write:

```
PUT(k, v2) succeeds
GET(k) immediately returns v2
```

Typical implementation:

- One leader per partition
- Majority replication
- Reads from leader
- Consensus for leader election and log ordering

Eventual consistency

A replica may temporarily return an older value:

```
Replica A: v2
Replica B: v1
Replica C: v2
```

Background repair eventually makes them equal.

Advantages:

- Better availability
- Lower geographic latency
- Reads from local replicas

Disadvantages:

- Stale reads
- Conflicting writes
- More difficult application semantics

Useful intermediate guarantees

A system can also support:

- Read-your-writes
- Monotonic reads
- Causal consistency
- Session consistency
- Linearizable conditional writes

---

10. Metadata and routing

Clients need to know which node owns a partition.

A metadata service can maintain:

```
Partition 0–99    → Node A
Partition 100–199 → Node B
Partition 200–299 → Node C
```

The routing process:

```
Client
  |
  ▼
Hash key to partition
  |
  ▼
Look up partition owner
  |
  ▼
Send request directly to node
```

There are two common models.

Proxy routing

```
Client → load balancer → coordinator → storage node
```

Advantages:

- Simple clients
- Centralized routing

Disadvantages:

- Additional network hop
- Coordinator can become a bottleneck

Client-side routing

```
Client SDK → correct storage node
```

Advantages:

- Lower latency
- Better scalability

Disadvantages:

- More complex SDK
- Partition metadata must be cached and refreshed

A practical design can support both: simple REST access through proxies and high-performance access through a smart SDK.

---

11. Rebalancing

When adding a new node:

```
Before:
Node A: P1 P2 P3 P4
Node B: P5 P6 P7 P8

After adding Node C:
Node A: P1 P2 P3
Node B: P5 P6 P7
Node C: P4 P8
```

Safe partition migration:

```
1. Mark partition as migrating
2. Copy a snapshot to the new node
3. Stream new writes to both nodes
4. Verify the copy
5. Switch routing metadata
6. Remove the old copy
```

Rebalancing should be throttled so it does not overwhelm user traffic.

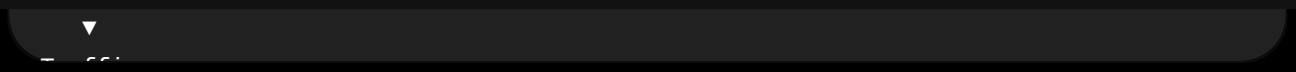
---

12. Handling failures

Node failure

The system detects failure using heartbeats.

```
Leader fails
   |
   ▼
Followers start election
   |
   ▼
New leader chosen
   |
   ▼
Metadata updated
   |
```



Disk failure

Replication allows rebuilding data from another node.

Network partition

The design must choose between:

- Continuing writes on one side
- Rejecting writes without quorum
- Allowing conflicts and reconciling later

For a strongly consistent design, only the majority side should accept writes.

Corrupted data

Use:

- Checksums per data block
- WAL checksums
- Periodic scrubbing
- Replica comparison
- Backups and snapshots

---

13. Delete handling

In an LSM tree, a delete does not immediately remove every old copy.

Instead, write a tombstone:

```
DELETE(customer:123)

customer:123 → TOMBSTONE
```

Reads treat the key as deleted. Compaction later removes the tombstone and older values.

Tombstones must be retained long enough to ensure stale replicas do not restore deleted data.

---

14. TTL support

Store an expiration timestamp with each value:

```
key: session:abc
value: ...
expires_at: 2026-07-29T12:00:00Z
```

Expiration can be handled through:

- Lazy deletion during reads
- Background expiration scans
- Compaction-time removal
- Timing wheels for high-volume TTL workloads

The read path should behave as if an expired key does not exist, even if it remains physically on disk.

---

15. Hot keys and hot partitions

A single popular key can overload one partition even with perfect hash distribution.

Examples:

```
GET(global_config)
GET/PUT/current_market_status)
```

Solutions:

- Cache hot reads at application or CDN level
- Replicate hot keys to additional read replicas
- Request coalescing
- Rate limiting
- Key splitting for counters
- Local in-process caching
- Dedicated partitions for known hot tenants

For a hot counter, instead of one key:

```
page_views = 10,000,000
```

use sharded counters:

```
page_views:0
page_views:1
...
page_views:99
```

Then sum the shards when reading.

---

16. Large values

Very large values should not necessarily be stored directly in the key-value engine.

Instead:

```
Key-value store:
document:123 → blob://bucket/abc123

Object storage:
blob://bucket/abc123 → large binary object
```

This keeps compaction, replication, caching, and memory usage manageable.

Set a maximum inline value size, for example 1 MB, depending on requirements.

---

17. Secondary features

A production system may additionally provide:

- Version numbers
- Compare-and-set
- Idempotency tokens
- Batch operations
- Compression
- Encryption
- Audit logging
- Multi-tenancy
- Backup and point-in-time recovery
- Change data capture
- Transactions over one key
- Limited transactions within one partition

Example compare-and-set:

```
Current:
account:123 = {status: OPEN, version: 7}

Request:
```

```
Update status to CLOSED only if version = 7

Result:
account:123 = {status: CLOSED, version: 8}
```

This prevents lost updates.

---

18. Capacity estimation example

Assume:

```
Number of keys: 10 billion
Average key size: 50 bytes
Average value size: 500 bytes
Metadata overhead: 100 bytes
Replication factor: 3
```

Logical storage:

```
10 billion × 650 bytes
≈ 6.5 TB
```

With replication:

```
6.5 TB × 3
≈ 19.5 TB
```

After compaction overhead, indexes, WALs, and operational headroom, provision perhaps:

```
30–40 TB
```

If one node safely stores 2 TB:

```
40 TB / 2 TB per node
≈ 20 storage nodes
```

You would add further spare capacity for failures and rebalancing.

---

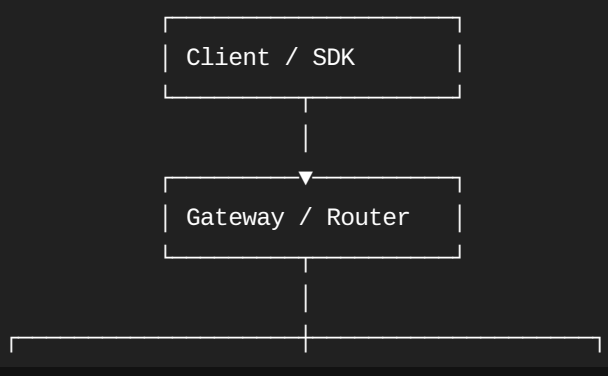
19. Recommended design

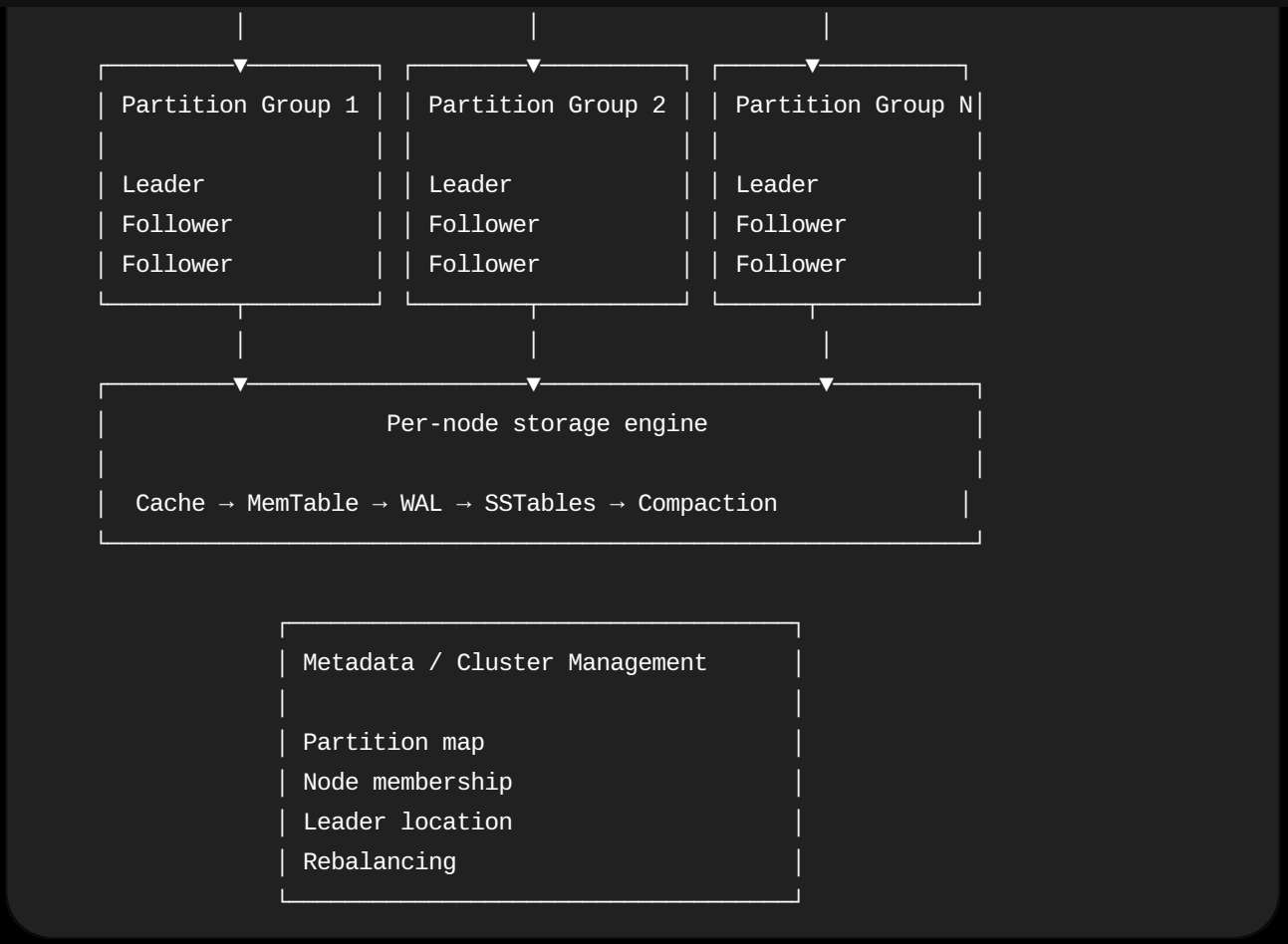
For a general-purpose durable key-value store, I would propose:

```
Partitioning:
- Hash-based logical partitions
- Thousands of partitions
- Metadata service maps partitions to nodes

Replication:
- Replication factor 3
- One Raft group per partition or partition group
- Leader-follower replication
- Majority acknowledgement

Storage:
- Write-ahead log
- In-memory MemTable
- Immutable SSTables
- Bloom filters
- Block cache
- Background compaction

Consistency:
- Strong consistency by default
- Leader reads
- Optional stale follower reads

Operations:
- Automatic leader election
- Incremental rebalancing
- Snapshots and backups
- Metrics, tracing, and alerts

Features:
- TTL
- Tombstones
- Compare-and-set
- Batch get/put
- Per-key version
```

---

20. Final architecture





For an interview, the most important design decisions to explain clearly are:

1. **Hash partitioning** distributes keys.
2. **Replication** protects against node failures.
3. **Raft and leaders** provide strong consistency.
4. **WAL + MemTable + SSTables** provide durable high-throughput storage.
5. **Compaction, rebalancing, TTL, and tombstones** make the system operationally complete.